Chuleta: Docker, Dockerfile y Docker Compose

Sistemas de Gestión Empresarial · Tema 0 · Incluye Prueba4 completa y errores típicos

0. Las piezas y cómo encajan

```
Dockerfile --docker build--> Imagen --docker run--> Contenedor
docker-compose.yml --docker compose up--> varios contenedores + red + volumen
```

Pieza	Una frase
Docker	La plataforma que lo ejecuta todo
Imagen	Plantilla de solo lectura (capas inmutables)
Contenedor	Instancia en ejecución de una imagen
Dockerfile	Receta para construir **una** imagen
Docker Compose	Fichero YAML para lanzar **varios** contenedores juntos

Regla de oro: `run` trabaja con imágenes y crea contenedores. `start`, `stop`, `exec`, `logs`, `rm` trabajan con contenedores que ya existen. `rmi` e `image` trabajan con imágenes.

1. Dockerfile

Instrucciones

Instrucción	Ejemplo	Qué hace	Fase
`FROM`	`FROM python:3.10-slim`	Imagen base (`nombre:etiqueta`)	Build
`WORKDIR`	`WORKDIR /usr/src/app`	Directorio de trabajo; si no existe se crea	Build

Instrucción	Ejemplo	Qué hace	Fase
`COPY`	`COPY requirements.txt app4.py ./`	Copia ficheros al contenedor; el último (`./`) es el destino (el WORKDIR)	Build
`ADD`	`ADD start.sh /start.sh`	Como COPY, pero admite URLs y descomprime `.tar`	Build
`RUN`	`RUN pip install --no-cache-dir -r requirements.txt`	Ejecuta un comando **al construir**	Build
`ENTRYPOINT`	`ENTRYPOINT ["python"]`	Ejecutable fijo	Runtime
`CMD`	`CMD ["app4.py"]`	Parámetros por defecto **al lanzar**; se pueden sobrescribir	Runtime
`ENV`	`ENV MENSAJE=HOLA`	Variable de entorno	Runtime
`USER`	`USER www-data`	Usuario del proceso	Runtime
`EXPOSE`	`EXPOSE 8080`	Solo **documenta** el puerto, no abre nada	Runtime
`VOLUME`	`VOLUME /datos`	Punto de montaje	Runtime
`LABEL`	`LABEL maintainer="a@b.com"`	Metadatos	Metadatos

Cosas que suelen preguntar

- **Build** (`docker build`) genera capas: `FROM` , `RUN` , `WORKDIR` , `COPY` , `ADD` . **Runtime** (`docker run`) define metadatos y comportamiento: `CMD` , `ENTRYPOINT` , `ENV` , `USER` , `EXPOSE` , `VOLUME` .
- `ENTRYPOINT` + `CMD` se juntan: `ENTRYPOINT ["python"]` + `CMD ["app4.py"]` ejecuta `python app4.py` . No repitas `python` en el CMD, o saldría `python python app4.py` .
- **Formato shell** (`CMD python app.py`): lanza `/bin/sh -c` , que toma el PID 1 y no reenvía SIGTERM. **Formato exec** (`CMD ["python","app.py"]`): recomendado; la app es el PID 1 y se apaga de forma ordenada.
- `EXPOSE` no publica nada; lo que publica el puerto es `-p host:contenedor` .
- `WORKDIR` encadenados con rutas relativas se suman (`/app` + `src` = `/app/src`); mejor rutas absolutas.

- `apt install -y`: el `-y` responde "sí" solo.

Dockerfile de Prueba4

```
FROM python:3.10-slim
WORKDIR /usr/src/app
COPY requirements.txt app4.py ./
RUN pip install --upgrade pip
RUN pip install --no-cache-dir -r requirements.txt
ENTRYPOINT ["python"]
CMD ["app4.py"]
```

2. Docker Compose

Estructura del fichero (3 bloques de nivel superior)

```
services:    # los contenedores
volumes:     # volúmenes que declaras
networks:    # redes que declaras
```

La clave `version:` es opcional (obsoleta en versiones modernas).

La regla que más falla: usar Y declarar

Todo lo que uses **dentro de un servicio** (volumen, red) tiene que estar **declarado también abajo del todo**, a 0 espacios:

Recurso	Se usa en el servicio	Se declara abajo
Volumen	`volumes: - dbdata4:/var/lib/mysql`	`volumes:` → `dbdata4:`
Red	`networks: - app4net`	`networks:` → `app4net:` → `driver: bridge`

- Si usas la red en un servicio y no la declaras abajo: **error**.
- Si la declaras abajo y no la usas en los servicios: existe, pero nadie la usa.
- **Red tipo bridge** = `driver: bridge` debajo del nombre de la red. Bridge es el tipo por defecto, pero el enunciado lo pide explícito.

Servicio vs contenedor

- `app4:` (bajo `services:`) es el **nombre del servicio**. Es el hostname que se usa dentro de la red (por eso `app4.py` conecta con `host='dbprueba4'`).
- `container_name: app4` es el **nombre real del contenedor** (el que sale en `docker ps -a`). Sin él, Compose pone uno automático tipo `proyecto-servicio-1`.
- El enunciado pide "nombre del servicio **y** contenedor": hay que poner las dos cosas.

`image` vs `build`

- `image: mysql:8.0` → descarga una imagen ya hecha de Docker Hub.
- `build: .` → construye la imagen con el Dockerfile de esa carpeta. El punto es el **directorio actual** (como el `./` de `docker build -t nombre ./`). `build` espera una carpeta, y dentro busca un fichero llamado exactamente `Dockerfile`.

Claves de un servicio

Clave	Ejemplo	Equivale en CLI	Qué hace
`image`	`image: mysql:8.0`	argumento IMAGEN	Imagen de Docker Hub
`build`	`build: .`	`docker build`	Construye con tu Dockerfile
`container_name`	`container_name: app4`	`--name`	Nombre del contenedor
`ports`	`- "3307:3306"`	`-p 3307:3306`	Puertos **host:contenedor**
`volumes`	`- dbdata4:/var/lib/mysql`	`-v`	Volumen **nombre:ruta**
`environment`	`MYSQL_ROOT_PASSWORD: luna`	`-e`	Variables de entorno
`env_file`	`- .env`		Variables desde fichero (no subir secretos a git)
`depends_on`	`- dbprueba4`		Orden de arranque; **no** espera a que esté listo

Clave	Ejemplo	Equivale en CLI	Qué hace
`restart`	`restart: always`	`--restart`	Política de reinicio
`networks`	`- app4net`	`--network`	Redes del servicio

`restart:` `no` (por defecto) · `always` (siempre) · `on-failure` (solo si sale con código distinto de 0) · `unless-stopped` (siempre, salvo `docker stop`).

Reglas del YAML

- **Solo espacios, nunca tabuladores.** Un tabulador da error.
- Sangría coherente (2 o 4 espacios por nivel). Recomendado: 2.
- Distingue mayúsculas y minúsculas.
- Las listas llevan guion (`- valor`) o corchetes (`["a","b"]`). Sin guion ni corchetes: error.
- `services`, `volumes` y `networks` van pegados al margen izquierdo (0 espacios).

3. Prueba4 completa

docker-compose.yml

```
services:
  dbprueba4:
    image: mysql:8.0
    container_name: dbprueba4
    environment:
      MYSQL_ROOT_PASSWORD: luna
      MYSQL_DATABASE: basedatos4
      MYSQL_USER: usuario4
      MYSQL_PASSWORD: luna
    ports:
      - "3307:3306"
    volumes:
      - dbdata4:/var/lib/mysql
    networks:
      - app4net

  app4:
```

```
    build: .
    container_name: app4
    depends_on:
      - dbprueba4
    networks:
      - app4net

volumes:
  dbdata4:

networks:
  app4net:
    driver: bridge
```

Pasos

1. `mkdir tunombre4` y `cd tunombre4`.
2. Dentro: `docker-compose.yml`, `Dockerfile`, `app4.py` (con **tu** nombre y apellidos) y `requirements.txt`.
3. `docker compose config` valida el YAML antes de lanzar.
4. `docker compose up --build` (sin `-d`, como pide el enunciado). Debe acabar con `app4 exited with code 0`.
5. En **otra terminal**: `docker exec -it dbprueba4 mysql -u usuario4 -p` (contraseña `luna`), luego `USE basedatos4;` y `SELECT * FROM employees;`.

Capturas para prueba4.pdf

Captura	Comando
Contenedores	`docker ps -a`
Imágenes	`docker images`
Red	`docker network ls`
Volumen	`docker volume ls`
Resultado de app4.py	la salida de `docker compose up`
Tabla en MySQL	`SELECT * FROM employees;`

Los nombres saldrán con prefijo del proyecto (`tunombre4_app4net` , `tunombre4_dbdata4`): es normal.

Checklist de errores típicos

- [ ] El servicio de la base de datos se llama `dbprueba4` (no `db`), porque `app4.py` conecta con ese `host` .
- [ ] `volumes:` y `networks:` declarados al final, a 0 espacios.
- [ ] `driver: bridge` bajo `app4net` .
- [ ] Sin tabuladores en el YAML.
- [ ] El fichero se llama exactamente `Dockerfile` (D mayúscula).
- [ ] `CMD ["app4.py"]` con `ENTRYPOINT ["python"]` , sin repetir `python` .
- [ ] Cambiaste el nombre en `app4.py` .
- [ ] Al principio puede salir "Reintentando en 5 segundos": es normal, MySQL tarda en arrancar.

4. Comandos de Docker (con parámetros)

docker run

`docker run [OPCIONES] IMAGEN [COMANDO]` — las opciones van **antes** de la imagen.

Parámetro	Significa
`-d`	Segundo plano (detached)
`-i`	Entrada abierta (interactive)
`-t`	Pseudo-terminal (tty)
`-it`	Sesión interactiva para entrar (p. ej. `bash`)
`--name x`	Nombre del contenedor
`-p 1200:80`	Puerto **host:contenedor**
`-e VAR=valor`	Variable de entorno
`-v origen:destino`	Volumen o bind mount
`--network red`	Conecta a una red

Parámetro	Significa
`--tmpfs /app`	Almacenamiento en RAM (se pierde al parar)
`--rm`	Borra el contenedor al salir

Contenedores

Comando	Qué hace
`docker ps` / `docker ps -a`	En ejecución / todos (`-a` = all, incluye parados)
`docker start x`	Arranca un contenedor parado
`docker start -ai x`	Lo arranca y conecta la terminal
`docker stop x`	Lo para
`docker rm x`	Lo borra
`docker exec -it x bash`	Terminal en un contenedor en marcha (varias sesiones posibles)
`docker exec -d x comando`	Comando en segundo plano
`docker attach x`	Se une al proceso principal (solo una sesión); salir sin pararlo: Ctrl+P, Ctrl+Q
`docker logs -n 10 x`	Últimas 10 líneas del log
`docker cp x:/ruta ./`	Copia entre contenedor y anfitrión

Un contenedor vive mientras viva su proceso principal. `Exited (0)` = terminó sin errores.

Imágenes

Comando	Qué hace
`docker images` (= `docker image ls`)	Lista imágenes
`docker pull alpine`	Descarga
`docker search ubuntu`	Busca en Docker Hub
`docker history alpine`	Capas de la imagen

Comando	Qué hace
`docker rmi ubuntu:14.04` (= `docker image rm`)	Borra una imagen (no se puede si un contenedor la usa)
`docker build -t nombre ./`	Construye desde un Dockerfile (`-t` = tag/nombre; `./` = carpeta)
`docker commit -m "msg" CONTENEDOR usuario/img:v`	Convierte un **contenedor** en imagen
`docker save -o f.tar imagen` / `docker load -i f.tar`	Exporta / importa (`-o` output, `-i` input)
`docker login` / `docker push usuario/img:v`	Subir a Docker Hub
`docker system prune -a`	Borra imágenes sin usar y contenedores parados

Redes

Comando	Qué hace
`docker network create redtest`	Crea una red bridge personalizada
`docker network create --driver bridge app4net`	Lo mismo, indicando el tipo
`docker network create --subnet=172.20.0.0/16 --gateway=172.20.0.1 mi-red`	Con subred y gateway
`docker network ls`	Lista redes
`docker network inspect redtest`	Detalles (contenedores, IPs)
`docker network rm redtest`	Borra la red
`docker network connect RED CONTENEDOR` / `disconnect`	Conecta / desconecta

Tipos de red:

- `bridge`: la por defecto (usa la interfaz `docker0`, rango `172.17.0.0/16`). En la bridge por defecto **no hay DNS por nombre**; en las que creas tú, **sí**.
- `host`: usa la IP y puertos del anfitrión; máximo rendimiento, sin aislamiento.
- `none`: aislamiento total.

Volúmenes

Tipo	Ejemplo
Bind mount	`-v /home/user/target:/app`
Volumen Docker	`-v micontenedor:/app`
tmpfs (RAM)	`--tmpfs /app`

`docker volume create/ls/rm nombre`. tmpfs: solo Linux, no se comparte, se pierde al parar.

5. Comandos de Compose

`docker compose` (v2, actual) · `docker-compose` (v1, antiguo, con guion). Se ejecutan desde la carpeta del YAML.

Comando	Qué hace
`docker compose up`	Crea y arranca todo mostrando logs
`docker compose up -d`	En segundo plano
`docker compose up -d --build`	Fuerza recompilar los Dockerfile
`docker compose up -d --scale web=3`	3 réplicas del servicio `web` (round-robin)
`docker compose -f otro.yml up -d`	Usa otro fichero YAML
`docker compose down`	Borra contenedores y redes; **mantiene** los volúmenes
`docker compose down -v`	Borra también los volúmenes
`docker compose down -t 5`	Espera 5 s antes de forzar
`docker compose ps`	Contenedores del proyecto

Comando	Qué hace
`docker compose logs` / `logs -f servicio`	Logs / seguir en vivo
`docker compose build` / `pull`	Construir / descargar imágenes
`docker compose stop` / `start`	Parar / arrancar sin borrar
`docker compose pause` / `unpause`	Congelar (SIGSTOP) / reanudar
`docker compose exec db bash`	Entrar en un contenedor en marcha
`docker compose run web npm test`	Contenedor **nuevo y efímero** para una tarea
`docker compose config`	Valida el YAML
`docker compose version`	Versión instalada

Detalles de Compose:

- Crea una red bridge por proyecto (`miproyecto_default`); los servicios se resuelven por **nombre de servicio** (`WORDPRESS_DB_HOST=db`).
- Antepone el nombre del proyecto a volúmenes y redes (`miproyecto_db_data`).
- `depends_on` : controla el **orden**, no que el servicio esté **listo** (MariaDB puede estar iniciada sin aceptar conexiones en el 3306).
- Buenas prácticas: secretos en `.env` (no en git), nombres explícitos para volúmenes, BD en redes internas, etiquetas concretas en vez de `latest` , imágenes Alpine.

6. Equivalencia CLI → Compose

```
docker run -d --name web -p 8080:80 -v datos:/data -e VAR=x --network red
nginx
```

```
services:
  web:
    image: nginx
    container_name: web
    ports: ["8080:80"]
    volumes: ["datos:/data"]
    environment:
      VAR: x
    networks: [red]
volumes:
  datos:
networks:
  red:
```

7. Trampas de examen

- `docker run` siempre necesita una **imagen**; `docker run Alfonso` (nombre de contenedor) da "invalid reference format".
- `-p host:contenedor`: el orden importa.
- `EXPOSE` no publica puertos.
- `depends_on` no espera a que esté listo.
- `docker rm` (contenedores) vs `docker rmi` (imágenes).
- `exec` crea un proceso nuevo; `attach` se conecta al principal.
- `docker compose down` mantiene los volúmenes; `down -v` los borra.
- Compose antepone el nombre del proyecto a redes y volúmenes.
- Los tabuladores están prohibidos en YAML.